dcFEM: Differentiable Finite Element Method

GSoC 2026 Proposal | DeepChem Organization

Candidate: Shriyam Baloni
University of Delhi (B.Tech CSE) | IIT Madras (B.Sc Data Science)
karanbaloni@gmail.com | github.com/CodeShrek

1 Abstract

dcFEM proposes a PyTorch-native, fully differentiable Finite Element Method (FEM) framework integrated into DeepChem. The primary objective is to deliver a **production-ready, reusable FEM utility** that enables end-to-end gradient flow through mesh representation, assembly, and solver stages. Unlike traditional FEM frameworks that rely on external C++ solvers (breaking the Autograd graph), dcFEM maintains the entire computational pipeline within PyTorch. This enables **physics-augmented learning**, gradient-based parameter inference, and hybrid ML–numerical workflows. The implementation includes a differentiable mesh data structure, sparse assembly using native PyTorch operations, and scalable iterative solvers. Integration with DeepChem's `TorchModel` and `GraphData` abstractions allows PDE-constrained learning within existing workflows.

Alignment with GSoC 2026 Expected Outcomes

- **Differentiable FEM Utility:** Delivered via a modular assembly and iterative solver pipeline (Section 3.2).
- **Mesh Data Structure:** Implementation of `FEMMesh` with direct `GraphData` interfacing (Section 3.3).
- **Benchmarks:** Rigorous validation against FEniCSx across Poisson and Reaction-Diffusion tasks (Section 7).

2 Problem Statement & Motivation

Scientific machine learning increasingly requires differentiable simulators for inverse problems and hybrid modeling. However, standard FEM tools (e.g., FEniCSx, Firedrake) are incompatible with PyTorch Autograd. DeepChem currently lacks a native PDE solver, limiting its capability in modeling transport phenomena and continuum mechanics. dcFEM addresses this gap by providing a native, differentiable FEM utility, enabling physics-augmented learning and inverse modeling workflows within DeepChem.

2.1 Positioning Relative to Prior Work

Traditional Neural FEM approaches (PINNs) often struggle with boundary accuracy and require retraining for each instance [1]. Neural Operators generalize better but are data-expensive. The Neural-Operator Element Method (NOEM) [2] uses NOs to accelerate subdomains. **dcFEM** differs by making the classical FEM formulation itself differentiable, enabling parameter recovery (inverse problems) within a numerical solver without requiring surrogate training data.

3 Technical Approach

3.1 Operator-Centric Design

PDEs will be constructed using modular, differentiable operators:

- **Diffusion:** $\nabla^2 u$ (Stiffness matrix assembly)
- **Advection:** $\mathbf{v} \cdot \nabla u$ (Non-symmetric transport terms)
- **Reaction:** $R(u)$ (Source/sink terms)

3.2 Differentiable Sparse Assembly and Solvers

Global system matrices (K) will be assembled using vectorized element-wise contributions. The global assembly is mathematically defined as:

$$K = \sum_e B_e^T D B_e \tag{1}$$

This is implemented via the PyTorch-native `index_add_` operation, ensuring that the assembly process is fully tracked by Autograd while remaining memory efficient through `torch.sparse_coo_tensor`.

Sparse Linear Solvers: dcFEM will implement iterative solvers compatible with sparse matrix-vector products:

- **Conjugate Gradient (CG):** For Symmetric Positive Definite systems (Poisson/Diffusion).
- **BiCGSTAB / GMRES:** For non-symmetric systems (Advection-Reaction).

Adjoint Sensitivity for Scalability: For large 3D meshes, dcFEM will implement a custom `torch.autograd.Function` using the **Adjoint Method**. By solving the adjoint system $K^T\lambda = \partial L/\partial u$, we compute gradients w.r.t. parameters without storing intermediate solver states, enabling memory-efficient training.

3.3 Mesh ↔ GraphData Integration

`FEMMesh` will bridge numerical simulation and DeepChem graph abstractions:

FEM Component	DeepChem Abstraction	Utility
Mesh Nodes	`GraphData.node_features`	Stores coordinates and gradients
Element Connectivity	`GraphData.edge_index`	Defines sparse sparsity patterns
Boundary Tags	`GraphData.node_features`	Masks for Dirichlet/Neumann BCs

3.4 Boundary Condition Handling

Penalty Method: Enforced via $\mathcal{L}_{BC} = \lambda\|u - u_{BC}\|^2$. Adaptive scaling of the penalty parameter λ will be used to avoid numerical stiffness, with **Direct Lifting** provided as an alternative for high-precision scenarios.

4 DeepChem Integration

- **PDEConstraintLoss:** A custom loss module: $\mathcal{L}_{total} = \mathcal{L}_{data} + \alpha\|K(p)u - f\|^2$.
- **FEMDataset:** Inherits from `dc.data.Dataset`, providing meshes and source terms as samples.
- **MeshLoader:** Utilities to import `.msh` (Gmsh) and `.vtk` files into `FEMMesh` objects.

5 Preliminary Work

I have implemented a prototype differentiable Poisson solver in PyTorch. The gradients have been validated against Finite Difference approximations with a relative error $< 10^{-6}$, and I have demonstrated successful recovery of an unknown diffusion coefficient D via gradient-based optimization.
Codebase: https://github.com/CodeShrek/dcfem-pytorch

6 Timeline (12 Weeks)

- **Weeks 1-2:** Implement `FEMMesh` core and element shape functions (1D/2D).
- **Weeks 3-4: Midterm Milestone:** Differentiable sparse assembly and CG solver.
- **Weeks 5-6:** Reaction-Diffusion and validation of non-symmetric solver (BiCGSTAB).
- **Weeks 7-8:** Adjoint Sensitivity module for memory-efficient backpropagation.
- **Weeks 9-11:** DeepChem Integration (`TorchModel` wrapper and `FEMDataset`).
- **Week 12:** Benchmarking vs FEniCSx and Final Documentation.

7 Benchmarks & Validation

Validation will be performed against **FEniCSx v0.7**+:

1. **Convergence:** Verify $O(h^k)$ error reduction with mesh refinement.
2. **Gradient Correctness:** Compare Autograd gradients vs. Finite Differences.
3. **Inverse Problem:** Recover a spatially varying diffusion field $D(x)$ from observed data.

8 Why Me

My research at **Universidad de Chile** involves coupling Stochastic Differential Equations (SDE) with PDEs to model cancer cell migration. I have previously implemented **Hidden Fluid Mechanics** frameworks for 3D Navier-Stokes equations and have extensive experience in PyTorch-based differentiable physics.

References

[1] Hildebrand, S., & Klinge, S. (2023). *Comparison of Neural FEM and Neural Operator Methods in Solid Mechanics*. arXiv:2307.02494.

[2] Ouyang, W., et al. (2025). *Neural-operator element method: Efficient and scalable FEM enabled by reusable neural operators*. arXiv:2506.18427.

Note on Authorship: This proposal was authored independently. All technical strategies and DeepChem integration plans were validated against the `deepchem/deepchem` repository (2026) to ensure originality and feasibility.